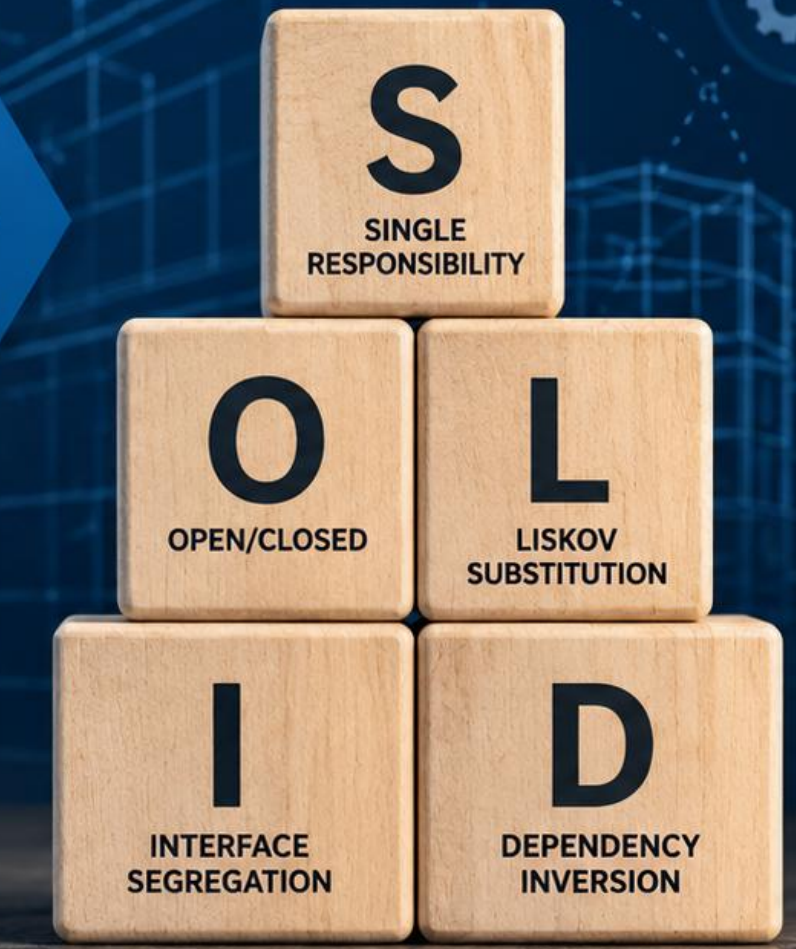
SOLID

DESIGN PRINCIPLES

AS DESIGN PATTERNS AND FRAMEWORKS

Build Flexible, Maintainable and Scalable Java Applications

S	O	L	I	D
Single Responsibility	Open/Closed Principle	Liskov Substitution	Interface Segregation	Dependency Inversion



Better Design. Better Patterns. Better Frameworks. Better Software.

LECTURE LEARNING OUTCOMES

By the end of the lecture, students should be able to:

01



Explain the **five SOLID principles**.

02



Identify violations of **SOLID principles** in Java code.

03



Refactor poorly designed **Java classes** using SOLID.

04



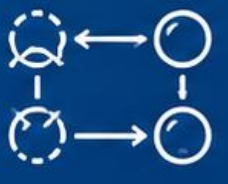
Explain the relationship between **SOLID** and **design patterns.**

05



Understand how **frameworks** use **SOLID** concepts.

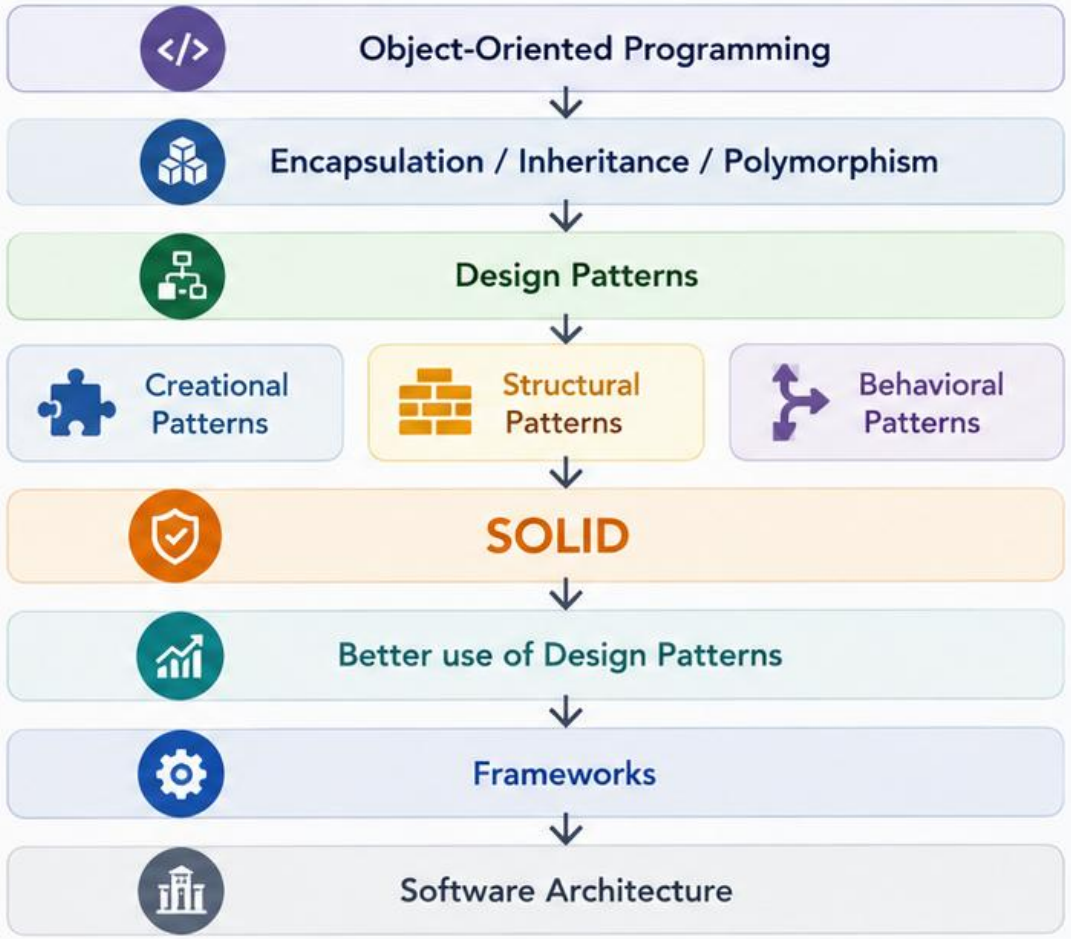
06



Recognize that SOLID is about **managing change** and **dependencies**, not simply writing more classes.



2. WHERE SOLID FITS IN THE DESIGN PATTERN SERIES



The big questions answered at each level

Design patterns answer:

"How can I **structure these objects** to solve this **recurring design problem**?"

SOLID answers:

"How should I design my **classes and dependencies** so that the system **remains flexible**?"

Frameworks answer:

"How can we provide a **reusable structure** for building an **entire category of applications**?"

Design Patterns solve recurring problems. **SOLID** keeps the solutions flexible.
Frameworks leverage these principles to build reusable application platforms.

The Five SOLID Principles

Guidelines for creating flexible, maintainable and change-friendly software

Principle	Main Idea	Key Question
S Single Responsibility	One reason to change	*What responsibility does this class have?*
O Open/Closed	Open for extension, closed for modification	*Can I add behavior without changing existing code?*
L Liskov Substitution	Subtypes must be substitutable	*Can I safely replace the parent with the child?*
I Interface Segregation	Small, focused interfaces	*Am I forcing clients to depend on methods they don't need?*
D Dependency Inversion	Depend on abstractions	*Does high-level code depend on concrete implementations?*

Memory Aid

Remember SOLID as:

- **S** ingle
- **O** pen
- **L** iskov
- **I** nterface
- **D** ependency

SOLID principles help us design classes and dependencies so that the system remains **flexible and easy to change.**

SRP — Bad Example

Single Responsibility Principle

Imagine an e-commerce system.

```
public class Order {

    public void calculateTotal() {
        // calculate order total
    }

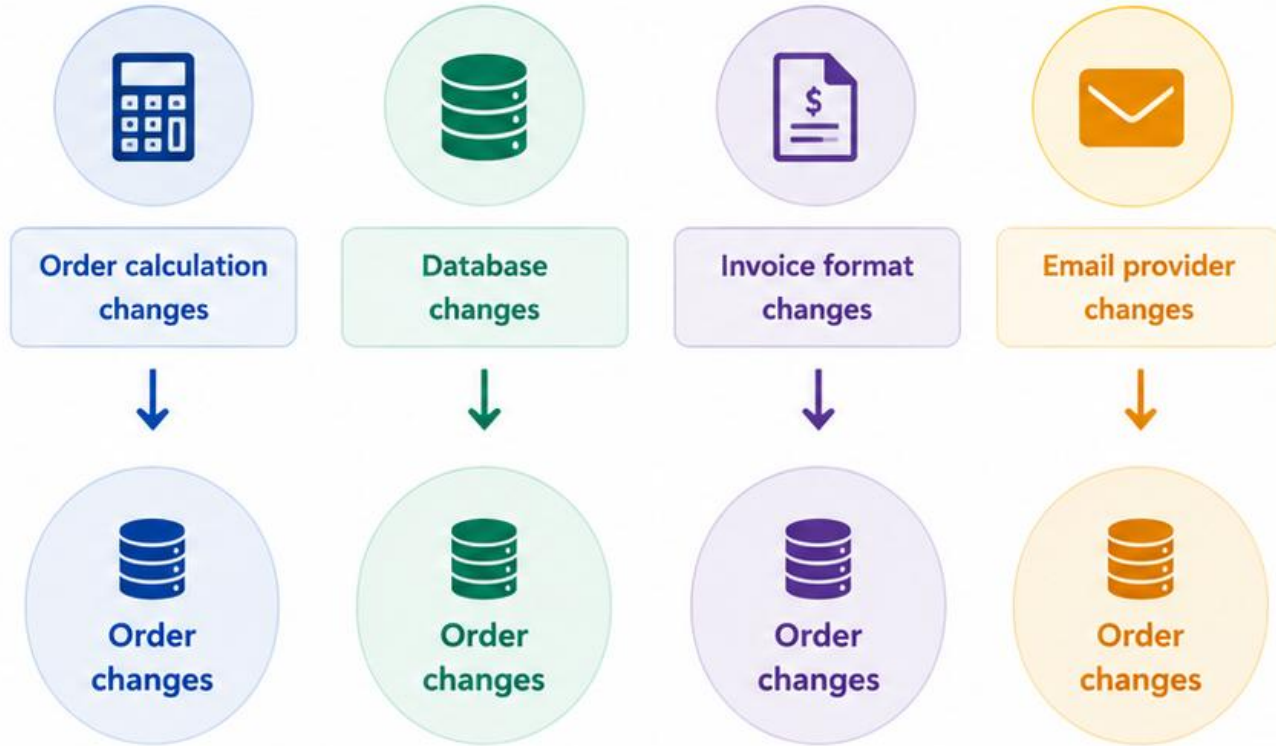
    public void saveToDatabase() {
        // save order
    }

    public void printInvoice() {
        // print invoice
    }

    public void sendEmail() {
        // send confirmation email
    }
}
```

At first glance this looks convenient.
But what happens when requirements change?

Possible reasons for changing Order



The Order class has **multiple responsibilities**.
A change in any one of these areas requires modifying the same class.

SRP — Refactoring

Single Responsibility Principle

Separate responsibilities into single-purpose classes.

1) Core Business Responsibility

```java
public class Order {
   private List<OrderItem> items;

   public double calculateTotal() {
      double total = 0.0;
      for (OrderItem item : items) {
         total += item.getPrice() * item.getQty();
      }
      return total;
   }
}
```

Business rules and calculations

2) Persistence Responsibility

```java
public class OrderRepository {
   private Connection connection;

   public void save(Order order) {
      // save order to database
      String sql = "INSERT INTO orders ...";
      // execute SQL
   }
}
```

Handles data storage and retrieval

3) Presentation Responsibility

```java
public class InvoicePrinter {

   public void print(Order order) {
      // print invoice
      // format and send to printer or PDF
   }
}
```

Handles invoice formatting and printing

4) Communication Responsibility

```java
public class EmailService {
   private EmailClient client;

   public void sendConfirmation(Order order) {
      // send email
      String subject = "Order Confirmation";
      client.send(order.getCustomerEmail(), subject);
   }
}
```

Handles email / communication

Now the responsibilities are separated



Benefits:

- ✓ Each class has one reason to change
- ✓ Easier to understand, test, and maintain
- ✓ Changes in one area don't affect others

Each class has a single, well-defined responsibility.

Changes in one area no longer force modifications to unrelated code.

SRP and Design Patterns

Single Responsibility Principle

SRP is strongly related to several design patterns.

Example Pattern: Facade

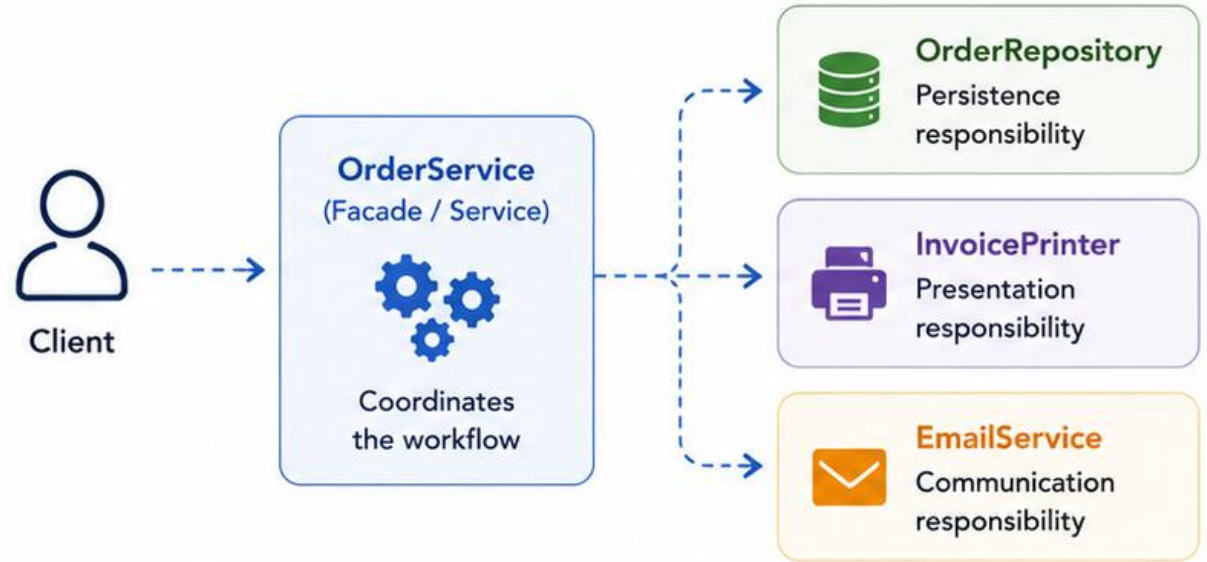
A facade can provide a simple interface while keeping responsibilities separated behind it.

```
public class OrderService {
    private OrderRepository repository;
    private EmailService emailService;
    private InvoicePrinter invoicePrinter;

    public void processOrder(Order order) {
        repository.save(order);
        invoicePrinter.print(order);
        emailService.sendConfirmation(order);
    }
}
```

The facade/service coordinates the workflow.
It does not need to perform every responsibility itself.

How it works



Why this supports SRP

- Each class has one reason to change.
- The service (facade) has a single responsibility: coordinating the use cases.
- Changes in persistence, email, or invoice formatting do not affect the **OrderService**.
- The system is easier to maintain, test, and extend.

SRP + Facade = Simple interface for clients, well-separated responsibilities inside. 🚀

SRP — Practical Question

Single Responsibility Principle

Goal: Help you quickly detect potential SRP violations in your classes.

1 Ask this question:

If the database technology changes, should the **Order** class change?

No

→ Good! Responsibilities are separated.
(The change is isolated in the persistence class.)

Yes

→ Warning! Potential SRP violation.
(Order is doing more than one thing.)

Why this works

A change in one reason (e.g., database technology) should not force a change in unrelated responsibilities.

2 Another useful question:

Can I describe what this class does using **"and"**?

Example:

This class calculates orders ***and*** saves orders ***and*** sends emails.

That's a warning sign.

You are likely mixing multiple responsibilities.

Remember

- A class should have **one reason** to change.
- If multiple changes in different areas can affect the class, it probably has **multiple responsibilities**.
- Keep classes **focused, cohesive**, and **easy to maintain**.

O — Open/Closed Principle

One of the Five SOLID Principles

Definition

Software entities should be **open for extension** but **closed for modification**.

This is one of the most important SOLID principles for understanding design patterns.

What it means

Existing tested code

- Well-tested
- Stable
- Working correctly

Keep it stable

→ **should**

We should not change what already works.

Avoid modifying existing code

→ **remain stable**

- No side effects
- No regressions
- No new bugs

Closed for modification

→ **New requirement**

A new feature or behavior is needed in the system.

Change is inevitable

→ **add new implementation**

Add new classes / implementations to extend behavior.

Open for extension

→ **rather than modify existing logic**

Extend the system without touching existing code.

Keep existing code closed to change

In short:

We **extend behavior** through **new code**, not by **changing** existing, tested code.

Benefits

- ✓ Easier to add new features
- ✓ Lower risk of introducing bugs
- ✓ Better maintainability and testability
- ✓ Supports flexible and scalable design

Key Takeaway

Be **open** to extend, but **closed** to change.

OCP — Bad Example

Open/Closed Principle

Problem: The class must be modified every time a new discount type is added.
That violates the **Open/Closed Principle**.

```java
public class DiscountCalculator {
    public double calculateDiscount(String customerType,
                                    double amount) {
        if (customerType.equals("REGULAR")) {
            return amount * 0.05;
        }
        if (customerType.equals("PREMIUM")) {
            return amount * 0.10;
        }
        if (customerType.equals("VIP")) {
            return amount * 0.20;
        }
        return 0;
    }
}
```

Initially, this works fine for a few types.
But what happens when we add more discount rules?

Suppose we add more discount types

- STUDENT
- CORPORATE
- EMPLOYEE
- LOYALTY
- BLACK_FRIDAY
- ...

What happens?

- We must modify the **DiscountCalculator** class every time.
- High risk of introducing bugs in existing logic.
- Requires re-testing existing code.
- Violates OCP: Not closed for modification.
- Harder to maintain as the number of types grows.

Key Takeaway

Every new requirement forces a change in existing, tested code. → **This increases risk and reduces maintainability.**

OCP — Using Polymorphism

Open/Closed Principle

> Software entities should be **open for extension** but **closed for modification.**

1 Create an abstraction

DiscountStrategy.java

```
public interfacs DiscountStrategy {
  double calculate(double amount);
}
```

The interface defines the contract. New discount types will implement this contract.

2 Implement different discount strategies

RegularDiscount.java

```
public class RegularDiscount
  implements DiscountStrategy {

  @Override
  public double calculate(double
    amount) {
      return amount * 0.05;
  }
}
```

PremiumDiscount.java

```
public class PremiumDiscount
  implements DiscountStrategy {

  @Override
  public double calculate(double
    amount) {
      return amount * 0.10;
  }
}
```

VipDiscount.java

```
public class VipDiscount
  implements DiscountStrategy {

  @Override
  public double calculate(double
    amount) {
      return amount * 0.20;
  }
}
```

Regular 5% discount

Premium 10% discount

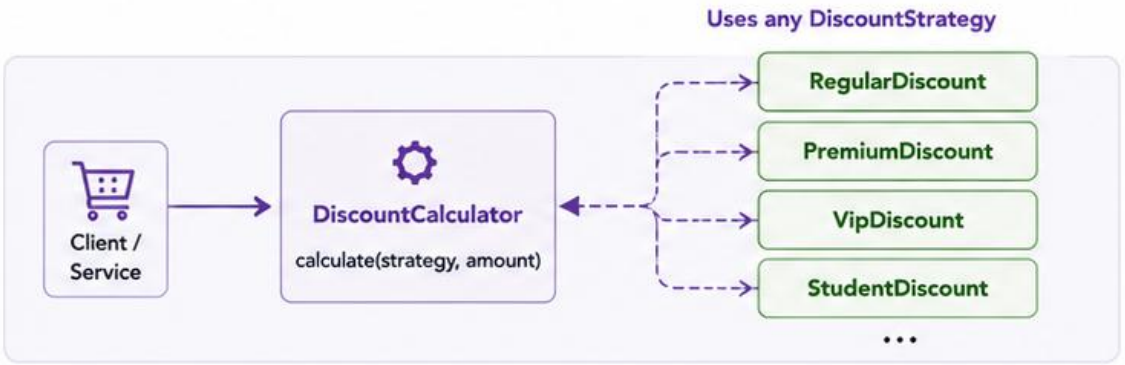
VIP 20% discount

3 Use the abstraction in the calculator

DiscountCalculator.java

```
public class DiscountCalculator {

  public double calculate(
      DiscountStrategy strategy,
      double amount) {
      return strategy.calculate(amount);
  }
}
```



4 Adding a new discount (extension)

StudentDiscount.java

```
public class StudentDiscount
  implements DiscountStrategy {

  @Override
  public double calculate(double
    amount) {
      return amount * 0.15;
  }
}
```

Student 15% discount

What happened?

- ✓ We added a new class.
- ✓ No changes to **DiscountCalculator**.
- ✓ Existing code remains working.
- ✓ System is **open for extension**, but **closed for modification.**

That's the Open/Closed Principle!

We achieve new behavior by **adding new implementations instead of modifying existing, tested code.**

Benefits

- ✓ Easy to add new discount types
- ✓ No risk to existing, tested code
- ✓ Better maintainability
- ✓ Follows OCP and supports SOLID design

→ Uses / depends on

⇢ Can use any implementation

OCP and Strategy Pattern

Open/Closed Principle in Action

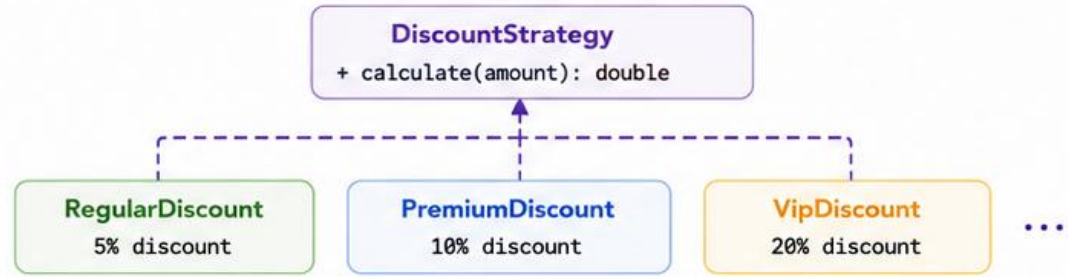
OCP Reminder

Software entities should be **open for extension** but **closed for modification.**

Goal:

Add new discount types without modifying existing, tested code.

Structure (Strategy Pattern)



How it works

1 Define the Strategy

```
public interface DiscountStrategy {
   double calculate(double amount);
}
```

Defines a common contract for all discount algorithms.

2 Create Concrete Strategies

```
public class RegularDiscount
        implements DiscountStrategy {
   public double calculate(double amount) {
       return amount * 0.05;
   }
}
```

```
public class PremiumDiscount
        implements DiscountStrategy {
   public double calculate(double amount) {
       return amount * 0.10;
   }
}
```

```
public class VipDiscount
        implements DiscountStrategy {
   public double calculate(double amount) {
       return amount * 0.20;
   }
}
```

Each class contains one version of the algorithm.

3 Context Uses the Strategy

```
public class DiscountCalculator {
    public double calculate(
        DiscountStrategy strategy,
        double amount) {
    return strategy.calculate(amount);
   }
}
```

DiscountCalculator does not know which strategy it uses.

It depends only on the abstraction (DiscountStrategy).

4 Strategies Become Interchangeable

```
DiscountStrategy strategy =
    new RegularDiscount();

DiscountCalculator calculator =
    new DiscountCalculator();

double discount =
    calculator.calculate(strategy, 1000);

// Change strategy at runtime
strategy = new VipDiscount();
discount = calculator.calculate(strategy, 1000);
```

We can switch strategies without changing the DiscountCalculator.

5 Add New Strategies (Extend, don't Modify)

```
public class StudentDiscount
       implements DiscountStrategy {
   public double calculate(double amount) {
       return amount * 0.15;
   }
}
```

We add new classes only. Existing code (e.g., DiscountCalculator) remains unchanged.

Why this achieves OCP

- ✓ New requirements are met by adding new strategy classes.
- ✓ Existing, tested code is not modified.
- ✓ Risk of introducing bugs in stable code is reduced.
- ✓ The system is easier to maintain and extend.

Strategy Pattern in a Nutshell

- Defines a family of algorithms.
- Encapsulates each algorithm.
- Makes them interchangeable.
- Lets the client choose the algorithm at runtime.

Key Takeaway

SOLID does not replace design patterns. SOLID helps us recognize design problems and decide when a design pattern may provide a good solution.

The Strategy Pattern is a common way to achieve the Open/Closed Principle.

OCP is the principle. Strategy is one possible solution.

Use the pattern when you have multiple interchangeable behaviors or algorithms that may change independently over time.

OCP Another Example — Payment Processing

Open/Closed Principle in Action

OCP: Be open for extension, but closed for modification. We should add new payment methods without changing existing, tested code.

Bad Design (Violates OCP)

Every new payment method requires **modification**.

PaymentProcessor.java

```java
public class PaymentProcessor {
    public void pay(String paymentType, double amount) {
        if (paymentType.equals("CARD")) {
            // process card
        } else if (paymentType.equals("PAYPAL")) {
            // process PayPal
        } else if (paymentType.equals("BANK")) {
            // process bank transfer
        }
    }
}
```

Problem

- To add a new payment method (e.g., CRYPTO), we must **modify** PaymentProcessor.
- Risk of introducing bugs in existing logic.
- Violates the **Open/Closed Principle**.

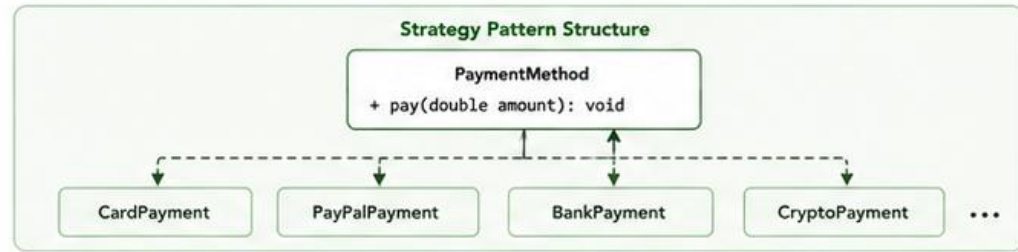
Better Design (Follows OCP with Strategy Pattern)

Add new payment methods by adding new classes, **not** by modifying existing code.

1 Abstraction

PaymentMethod.java

```java
public interface PaymentMethod {
    void pay(double amount);
}
```



2 Concrete Implementations

CardPayment.java

```java
public class CardPayment
    implements PaymentMethod {
  @Override
  public void pay(double amount) {
    System.out.println(
      "Card payment: " + amount);
  }
}
```

PayPalPayment.java

```java
public class PayPalPayment
    implements PaymentMethod {
  @Override
  public void pay(double amount) {
    System.out.println(
      "PayPal payment: " + amount);
  }
}
```

BankPayment.java

```java
public class BankPayment
    implements PaymentMethod {
  @Override
  public void pay(double amount) {
    System.out.println(
      "Bank transfer: " + amount);
  }
}
```

NEW: CryptoPayment.java (New Extension)

```java
public class CryptoPayment
    implements PaymentMethod {
  @Override
  public void pay(double amount) {
    System.out.println(
      "Crypto payment: " + amount);
  }
}
```

3 Processor (unchanged when new methods are added)

PaymentProcessor.java

```java
public class PaymentProcessor {
    public void process(PaymentMethod paymentMethod, double amount) {
        paymentMethod.pay(amount);
    }
}
```

Benefit

- Add new payment methods by creating new classes.
- No changes to PaymentProcessor.
- Existing code remains stable and tested.
- Follows **Open/Closed Principle**.

Key Takeaway: Use polymorphism (**Strategy Pattern**) to support new behavior without modifying existing, tested code.

Closed for modification → Open for extension

L — Liskov Substitution Principle

One of the Five SOLID Principles

Definition

Objects of a superclass should be **replaceable** with objects of its **subclasses** without breaking the **correctness of the program.**

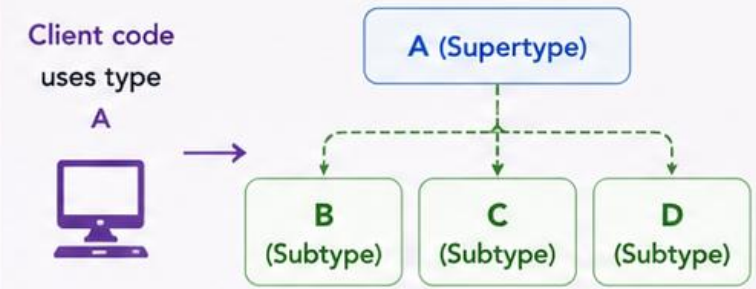
In simple terms

If B is a subtype of A, then B should **behave like an A** where **A is expected.**

What does it mean?

- A subclass must honor the contract of its superclass.
- It must not change the expected behavior.
- It must not weaken preconditions.
- It must not strengthen postconditions.
- It must not throw new exceptions that the superclass does not throw.

The Substitution Test



Why is it Important?

- Ensures reliability and correctness.
- Enables polymorphism to work safely.
- Prevents surprising behaviors.
- Improves maintainability and extensibility of code.

If LSP is Violated

- Substituting a subclass breaks the program.
- Leads to runtime errors or incorrect results.
- Destroys trust in inheritance and polymorphism.

Bad Outcome

```
List<A> list = ...;
list.add(new B());   // B is-a A
for (A a : list) {
    a.someOperation();  // Unexpected behavior!
}
// The program breaks or returns wrong result.
```

Good Outcome (LSP Respected)

```
List<A> list = ...;
list.add(new B());   // B is-a A
for (A a : list) {
    a.someOperation();   // Works as expected
}
// Substitution works. Program remains correct.
```

Key Takeaway

Design subclasses that truly behave like their superclasses.

When in doubt, ask: **"Can I replace the superclass object with this subclass object without breaking anything?"**

Think: **Behavior** over inheritance.

LSP — Classic Rectangle Example

Liskov Substitution Principle

LSP: Subtypes must be substitutable for their base types without changing the correctness of the program.

1 Rectangle (Supertype)

```
public class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}
```

2 Square (Subtype)

```
public class Square extends Rectangle {

    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;  // keep it a square
    }

    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height;  // keep it a square
    }
}
```

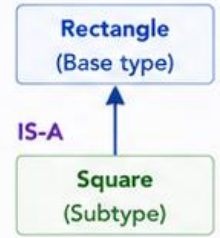
A Square overrides setWidth and setHeight to keep both sides equal.
This changes the behavior expected by clients of Rectangle.
Therefore, Square is not a valid substitute for Rectangle.

Mathematically, a square IS a rectangle.
But **behaviorally**, Square **breaks the expectations** of Rectangle.

3 Using a Square where a Rectangle is expected

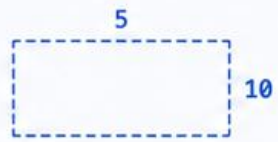
```
Rectangle rectangle = new Square();
rectangle.setWidth(5);    // Expect width = 5
rectangle.setHeight(10);  // Expect height = 10

System.out.println(rectangle.getArea());
```



Expected by Rectangle contract

Width = 5, Height = 10



Area = 5 × 10 = 50

≠

Actual result with Square

Square forces both sides to 10



Area = 10 × 10 = 100

The subtype (Square) changes the meaning of the operations inherited from Rectangle.

LSP is violated.

Key Takeaways

- Inheritance does not automatically imply Liskov substitutability.
- Subtypes must honor the contracts (behavior and expectations) of their supertypes.
- If a subtype changes the meaning of inherited operations, it violates LSP.

Good Design Tip

Prefer composition over inheritance when the subtype cannot truly honor the behavior of the supertype.

LSP — Better Design

Design abstractions that allow safe substitution.

Bad Design: Forcing a Square into Rectangle

Square is-a Rectangle? Not really. It violates LSP.

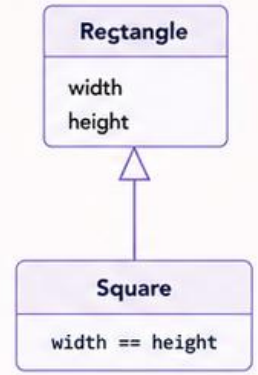
```
public class Rectangle {
   protected int width;
   protected int height;
   public void setWidth(int width) { this.width = width; }
   public void setHeight(int height) {  this.height = height; }
   public int getArea() {  return width * height; }
}
public class Square extends Rectangle {
    public Square(int side) {
        this.width = side;
        this.height = side;
    }
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;   // forces height to change!
    }
    @Override
    public void setHeight(int height) {
        this.width = height;  // forces width to change!
        this.height = height;
    }
}
```

Why this breaks LSP

Client code expects that changing width does not affect height.

```
Rectangle r = new Square(5);
r.setWidth(10);
// Client expects: width = 10, height = 5
// Reality: width = 10, height = 10
```

Substituting **Square** where **Rectangle** is expected breaks the expected behavior.



Forcing a square into rectangle hierarchy causes incorrect behavior.

Key Idea

Don't force a class into a hierarchy if it doesn't truly follow the substitutability contract.

LSP Requires

- Same contract
- Same expectations
- No stronger preconditions
- No weaker postconditions
- No surprising behavior

Better Design: Use a Common Abstraction

Both **Rectangle** and **Square** are **Shapes**.
They can be used interchangeably where **Shape** is expected.

Shape.java

```
public interface Shape {

   int getArea();
}
```

Rectangle.java

```
public class Rectangle implements Shape {
   private int width;
   private int height;

   public Rectangle(int width, int height) {
      this.width = width;
      this.height = height;
   }

   @Override
   public int getArea() {
      return width * height;
   }
}
```

Square.java

```
public class Square implements Shape {
   private int side;

   public Square(int side) {
      this.side = side;
   }

   @Override
   public int getArea() {
      return side * side;
   }
}
```

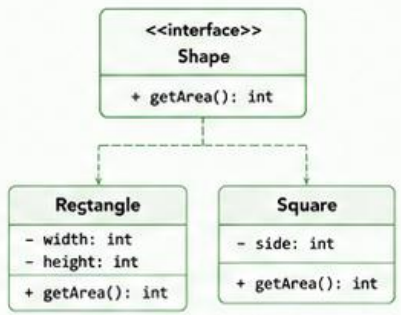
Usage

```
Shape shape;

shape = new Rectangle(4, 6);
System.out.println(shape.getArea()); // 24

shape = new Square(5);
System.out.println(shape.getArea()); // 25

// Client code works with Shape
// without knowing the concrete type.
```



Both implement Shape.
Either can be used wherever a Shape is expected.

Benefits

- ✓ Respects LSP
- ✓ No surprising behavior
- ✓ Easier to extend
- ✓ Cleaner, more maintainable code
- ✓ Focus on behavior, not unnecessary inheritance

Takeaway

Model the right abstraction.
If two classes share behavior (e.g., getArea), use a common abstraction (Shape), not an inheritance relation that forces incorrect assumptions.

Good design enables safe substitution.
That is the essence of Liskov Substitution Principle.

> If B is a subtype of A,
> then B should behave like an A
> where A is expected.

LSP — Real-World Example (Bird and Penguin)

Liskov Substitution Principle

LSP: Subtypes must be substitutable for their base types without changing the correctness of the program.

1 Base Type: Bird

```
public class Bird {
   public void fly() {
      System.out.println("Flying");
   }
}
```

The Bird contract says: Every Bird can fly.

2 Subtype: Penguin

```
public class Penguin extends Bird {

  @Override
  public void fly() {
     // Penguins cannot fly!
     throw new
        UnsupportedOperationException(
           "Penguins cannot fly");
  }
}
```

Penguin breaks the expectation of the Bird contract.

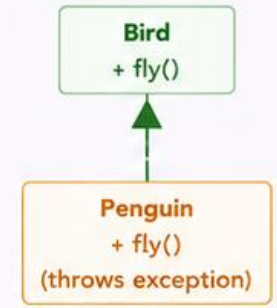
Problem: LSP is violated

Clients that work with `Bird` objects expect `fly()` to work.

3 Client Code — Expects fly() to work for any Bird

```
public class BirdClient {
   public void makeItFly(Bird bird) {
      bird.fly();     // Client expects this to work
   }

   public static void main(String[] args) {
      Bird bird = new Penguin();    // Substituting Penguin for Bird
      BirdClient client = new BirdClient();
      client.makeItFly(bird);       // BOOM at runtime!
   }
}
```



Runtime Result

```
Exception in thread "main" java.lang.UnsupportedOperationException: Penguins cannot fly
      at Penguin.fly(Penguin.java:5)
      at BirdClient.makeItFly(BirdClient.java:4)
}     at BirdClient.main(BirdClient.java:9)
```



I'm a Bird, but I can't fly! Client is surprised!

Why LSP is violated here?

- The base type (`Bird`) promises that `fly()` is a valid operation.
- The subtype (**Penguin**) **cannot** fulfill that promise.
- Clients using the base type are broken by the subtype.

Key Insight

Inheritance does not automatically imply Liskov substitutability.

A subtype must honor the behavior and expectations defined by its supertype.

If a subclass cannot do what the parent type promises, it is not a valid substitute.

How to Fix?

Do not put `fly()` in a class that includes birds that cannot fly.
Use better abstractions:

- Seperate **Flyable** behavior from **Bird**.
- Apply composition or interfaces.

Key Takeaways

- ✓ LSP ensures that programs remain correct when using subtypes.
- ✓ Subtypes must be able to do everything their supertypes promise.
- ✓ Breaking expectations leads to runtime errors and fragile designs.

> If it looks like a **Bird**,
> acts like a Bird,
> it must truly be able to fly
> whenever the client asks it to.

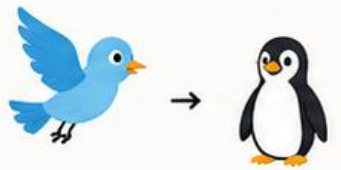
LSP — Real-World Example

Liskov Substitution Principle

Definition: Objects of a superclass should be **replaceable** with objects of its **subclasses** without breaking the **correctness of the program.**

What happened?

1. Bird promises it can fly.
2. Penguin is a Bird (subtype).
3. But Penguin cannot actually fly.
4. Substituting Penguin where Bird is expected breaks the program.



Bird Can fly → **Penguin** Cannot fly!

Problem: LSP Violation

Penguin is a Bird, **but it cannot fly.**
It breaks the expectation of the client.

Bird.java

```
public class Bird {

   public void fly() {

       System.out.println("Flying");
   }
}
```

Penguin.java

```
public class Penguin extends Bird {

   @Override
   public void fly() {

      throw new
          UnsupportedOperationException(
      |    "Penguins cannot fly");
   }
}
```

Client.java

```
public class Client {

   public static void main(String[] args) {
      Bird bird = new Penguin();

      // Client expects fly() to work
      bird.fly();   // Runtime exception!
   }
}
```

Output (Runtime)

```
Exception in thread "main" java.lang.UnsupportedOperationException:
Penguins cannot fly
```

Why is this a problem?

- The subclass (Penguin) **breaks** the behavior expected from the **superclass** (Bird).
- The client code **fails at runtime.**
- This **violates LSP.**

Better Design: Use a More Appropriate Abstraction

Not all birds can fly. So don't put fly() in the base class.
Separate the capability.

Bird.java

```
public abstract class Bird {

   public void eat() {

       System.out.println("Eating");
   }
}
```

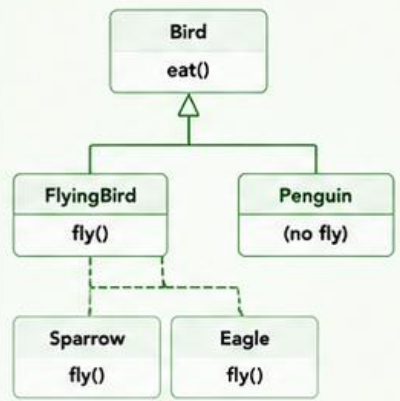
FlyingBird.java

```
public abstract class FlyingBird extends Bird {

    public abstract void fly();
}
```

Penguin.java

```
public class Penguin extends Bird {
    // No fly() here
    // Penguins don't fly
}
```



Client.java

```
public class Client {
   public static void main(String[] args) {
       Bird b1 = new Penguin();
       b1.eat();              // OK

       FlyingBird b2 = new Sparrow();
       b2.fly();              // OK

       // Now substritution is always safe
   }
}
```

Benefits

- ✓ Respects LSP
- ✓ No surprising behavior
- ✓ Clients don't break
- ✓ More realistic model
- ✓ Easier to extend

Key Takeaway Subtypes must be substitutable for their base types without breaking expected behavior.

If a subclass cannot fulfill the contract of the superclass, your design needs to change.

> If B is a subtype of A, then B should behave like an A where A is expected.

LSP — Better Bird Design

Design by separating capabilities, not by forcing inheritance.

Principle: Subtypes must be substitutable for their base types without changing the correctness of the program.

Problem: LSP is violated

The base type `Bird` promises that every bird can fly. `Penguin` breaks that promise.

```
// Bad Design
public class Bird {
   public void fly() {
      System.out.println("Flying");
   }
}
```

```
public class Penguin extends Bird {
   @Override
   public void fly() {
      throw new UnsupportedOperationException(
            "Penguins cannot fly");
   }
}
```

```
// Client Code
Bird bird = new Penguin();    // Looks fine
bird.fly();                   // BOOM at runtime!
```

What goes wrong?

- Client expects `fly()` to work for any `Bird`.
- Penguin cannot keep that contract.
- This breaks **Liskov Substitution Principle.**

Better Design: Separate capabilities

We only promise `fly()` for birds that can actually fly.

1 Base Abstraction

```
public interface Bird {
   // Marker interface
   // Represents the idea of a Bird
}
```

`Bird` is a general type. It does not promise flying.

2 Flying Capability

```
public interface FlyingBird
      extends Bird {
   void fly();
}
```

Only birds that can fly implement this interface.

3 Sparrow (Can Fly)

```
public class Sparrow
      implements FlyingBird {
   @Override
   public void fly() {
      System.out.println(
         "Sparrow flying");
   }
}
```

Sparrow is a `FlyingBird`, so it must implement `fly()`.

4 Penguin (Cannot Fly)

```
public class Penguin
      implements Bird {
   // Penguins don't fly.
   // No fly() here.
}
```

Penguin is a `Bird`, but not a `FlyingBird`.

5 Client Usage — Now Safe and Correct

Client working with any Bird (general)

```
public void printBirdInfo(Bird bird) {
   System.out.println("I am a bird.");
   // We don't assume it can fly.
}
```

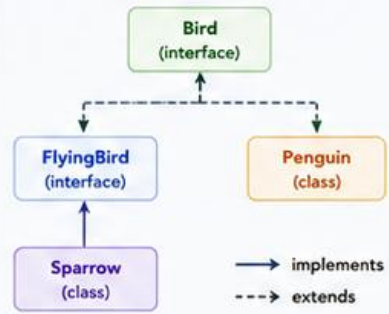
Works for any `Bird` (Penguin, Sparrow, etc.).

Client working with birds that can fly

```
public void makeItFly(FlyingBird bird) {
   bird.fly();    // Safe: guaranteed to exist
}
```

Only `FlyingBirds` can be passed here.

Type Relationship



Real World



Sparrow can fly.



Penguin is a bird, but it cannot fly.

Key Takeaway

- ✓ Don't put operations in a base type that not all subtypes can support.
- ✓ Model capabilities using interfaces (or abstract classes).
- ✓ This keeps contracts honest and your code robust.

Result

- No broken promises.
- No runtime surprises.
- LSP is respected.

Remember

"If it looks like a Bird, acts like a Bird, it must be able to do what a Bird is expected to do."

If not, don't make it a Bird.

LSP — Important Teaching Point

Liskov Substitution Principle

> Objects of a superclass should be **replaceable** with objects of its **subclasses** without breaking the **correctness** of the program.

Tell students:

✗ Don't ask only:

"Is-a relationship?"

✓ Also ask:

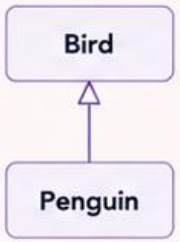
"Can the child fulfill the behavioral contract of the parent?"

This is a much **deeper understanding** of inheritance.

Inheritance is not just about structure, it's about behavior

Only "Is-a"?
(Too shallow)

Looking only at the type hierarchy.



Yes, Penguin is-a Bird.

VS

Behavioral Contract?
(The real question)

Looking at what the parent promises, and whether the child can deliver.

✓ Bird promises: can fly()

✗ Penguin cannot fly()

So, Penguin breaks the contract.

Think Beyond Types

A subtype must **honor the expectations (behavior)** established by its supertype.

Example: Bird and Penguin

Bird.java

```
public class Bird {
  public void fly() {
     System.out.println("Flying");
  }
}
```

Bird's contract:
I can fly.

Penguin.java

```
public class Penguin extends Bird {
   @Override
   public void fly() {
      throw new
       UnsupportedOperationException(
   }     "Penguins cannot fly");
}
```

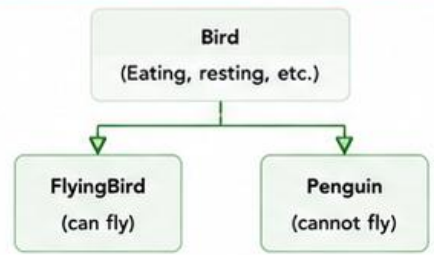
Penguin breaks the contract:
I cannot fly.

Client.java

```
public class Client {
   public static void main(String[] args) {
      Bird b = new Penguin();
      b.fly();   // Client expects fly()
   }
}
```

Result at runtime:
Exception! Contract broken.

Good Design: Use the Right Abstraction



✓ Clients that need flying behavior depend on FlyingBird.

✓ Penguin is still a Bird, but not forced to implement something it cannot do.

✓ **Substitution is safe because the contract is respected.**

Key Takeaway

Inheritance is **a promise**. If the child cannot keep that promise, substitution will break, and your design will too.

> Good inheritance is not about reusing code, it's about **preserving meaning and behavior**.

I — Interface Segregation Principle

Definition

Clients should not be forced to depend on methods they do not use.

In Simple Terms

Prefer several small, focused interfaces over one large interface.

Why does ISP matter?

- Reduces unnecessary dependencies.
- Makes code easier to understand and maintain.
- Changes in one part do not affect unrelated clients.
- Leads to more flexible and reusable designs.

Bad Example — Fat Interface

Clients are forced to depend on methods they do not use.

Large Interface

```
public interface Worker {
    void work();
    void eat();
    void sleep();
    void drive();
    void code();
}
```

Robot (only works)

```
public interface implements Worker {
    pubic void work() { ... }
    // eat() not used
    // sleep() not used
    // driep() not used
    // code() not used
}
```

Driver (only drives)

```
public class Driver implements Worker {
    public void drive() { ... }
    // work() not used
    // eat() not used
    // sleep() not used
    // code() not used
}
```

If Worker changes (e.g., a new method is added), all clients are affected even if they don't use it.

Better Example — Segregated Interfaces

Each client **depends** only on the methods it **actually uses**.

Workable

```
public interface Workable {
    void work();
}
```

Eatable

```
public interface Eatable {
    void eat();
}
```

Drivable

```
public interface Drivable {
    void drive();
}
```

Codable

```
public interface Codable {
    void code();
}
```

Robot

```
public class Robot
    implements Workable {
  public void work() { ... }
}
```

Driver

```
public class Driver
    implements Drivable {
  public void drive() { ... }
}
```

Developer

```
public class Developer
    implements Workable,
               Eatable,
               Coddale {
  public void work() { ... }
  public void eat() { ... }
  public void code() { ... }
}
```

Benefits of ISP

- ✓ Clients depend only on what they use.
- ✓ Interfaces are smaller and easier to implement.
- ✓ Code is easier to maintain, extend and test.
- ✓ Promotes loose coupling and high cohesion.

Key Takeaway

Don't make clients depend on things they don't need.
Many small interfaces are better than one big interface.

ISP — Bad Example

Interface Segregation Principle

Problem: The interface is too large and forces classes to depend on methods they do not need or cannot use.

Employee.java

```
public interface Employee {
    void work();
    void eat();
    void sleep();
    void attendMeeting();
    void writeCode();
}
```

Many methods that not every implementer can or should have.

Robot.java

```
public class Robot implements Employee {

    @Override
    public void work() {
        System.out.println("Robot working");
    }

    @Override
    public void eat() {
        throw new UnsupportedOperationException();
    }

    @Override
    public void sleep() {
        throw new UnsupportedOperationException();
    }

    @Override
    public void attendMeeting() {
        throw new UnsupportedOperationException();
    }

    @Override
    public void writeCode() {
        throw new UnsupportedOperationException();
    }
}
```

What's Wrong?

- Robot is forced to implement methods it does not support.
- Leads to UnsupportedOperationException at runtime.
- Violates ISP: clients (classes) depend on methods they don't use.
- Harder to maintain and extend.
- Any change to this large interface affects all implementers.

Real-World Analogy

It's like forcing a printer to "eat", "sleep" and "attend meetings".

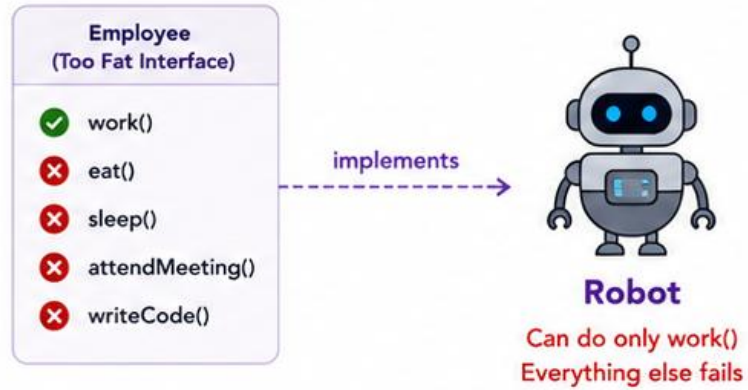
It doesn't make sense!



ISP Reminder

Clients should not be forced to depend on interfaces they do not use. Keep interfaces small and focused.

Visual Impact



Runtime Result

```
Robot robot = new Robot();
robot.work();          // OK
robot.eat();          // Throws Exception!

Exception in thread "main" java.lang.UnsupportedOperationException
    at Robot.eat(Robot.java:12)
    at Main.main(Main.java:7)
```

Key Takeaway

Large, "fat" interfaces are a bad design.
Break interfaces into smaller, role-specific ones.

Goal of ISP

Small interfaces, clients only depend on what they actually need.
Better design, fewer bugs, easier to maintain.

ISP — Interface Segregation Principle

Definition: Clients should not be forced to depend on methods they do not use.

In Simple Terms:
Prefer several small, focused interfaces over one large interface.

Why ISP?

- Reduces unnecessary dependencies
- Makes code easier to understand and maintain
- Changes in one interface don't affect unrelated clients
- Leads to more flexible and reusable designs

Bad Design — Fat Interface

Clients are forced to depend on methods they do not use.

Fat Interface

```
public interface Worker {
    void work();
    void eat();
    void sleep();
}
```

- Not all workers eat (e.g., robots)
- Not all workers sleep (e.g., robots)
- Forces all classes to implement everything

Human implements Worker

```
public class Human implements Worker {

    @Override
    public void work() {
        System.out.println("Human working");
    }
    @Override
    public void eat() {
        System.out.println("Human eating");
    }
    @Override
    public void sleep() {
        System.out.println("Human sleeping");
    }
}
```

Robot implements Worker

```
public class Human implements Worker {

    @Override
    public void work() {
        System.out.println("Robat working");
    }
    @Override
    public void eat() {
        // Not applicable for Robot
        throw new UnsupportedOperationException();
    }
    @Override
    public void sleep() {
        // Not applicable for Robot
        throw new UnsupportedOperationException();
}
```

Robot is forced to depend on `eat()` and `sleep()`, which it does not need.
This violates the Interface Segregation Principle.

Better Design — Segregated Interfaces

Each client **depends** only on the methods it **actually uses**.

Workable

```
public interface Workable {
    void work();
}
```

Eatable

```
public interface Eatable {
    void eat();
}
```

Sleepable

```
public interface Sleepable {
    void sleep();
}
```

Human implements all capabilities

```
public class Human
    implements Workable, Eatable, Sleepable {

    @Override
    public void work() {
        System.out.println("Human working");
    }
    @Override
    public void eat() {
        System.out.println("Human eating");
    }
    @Override
    public void sleep() {
        System.out.println("Human sleeping");
    }
}
```

Robot implements only what it needs

```
public class Robot implements Workable {

    @Override
    public void work() {
        System.out.println("Robot working");
    }
}
```

Robot depends only on `Workable`.
It is not forced to implement `eat()` or `sleep()`.
This follows the Interface Segregation Principle.

Key Takeaway

Don't depend on things you don't use. Small, specific interfaces lead to better, safer, and more maintainable code.

Fat Interface → Small Interfaces → Better Design

ISP and Java Interfaces

Interface Segregation Principle in Action

Common Misconception

Students often think:

```
interface = class with abstract methods
```

Why this is limiting

- Leads to large ("fat") interfaces
- Forces classes to depend on methods they don't need
- Violates ISP
- Harder to design, implement, and maintain

The Right Mental Model

An interface represents a **capability** or **contract**.

A class implements an interface if it **has** that capability and **can fulfill** the contract.

What is a Capability?

A capability is something a type can do.
Interfaces capture these "can do" abilities.



Comparable
can be compared



Runnable
can be run



Iterable
can be iterated



AutoCloseable
can be closed

Bad vs Good Interface Design

Bad (Fat Interface)

```
interface Worker {
    void work();
    void eat();
    void sleep();
    void attendMeeting();
    void writeCode();
}
```

Too many methods. Not every class needs all of these.

Good (Role-Specific Interfaces)

```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

interface Sleepable {
    void sleep();
}

interface MeetingParticipant {
    void attendMeeting();
}

interface CodeWriter {
    void writeCode();
}
```

Small, focused, and role-specific.

Clients depend only on the interfaces they need.

Real Java Interfaces = Capabilities

The Java standard library is full of capability-oriented interfaces.
They don't force classes to be or do everything.



Comparable<T>
Capability to be compared with others.

```
public interface Comparable<T> {
    int compareTo(T other);
}
Example: String, Integer, LocalDate
```



Runnable
Capability to be executed by a thread.

```
public interface Runnable {
    void run();
}
Example: Thread, Executors
```



Iterable<T>
Capability to provide an iterator over elements.

```
public interface Iterable<T> {
    Iterator<T> iterator();
}
Example: List, Set
```



AutoCloseable
Capability to be closed (release resources).

```
public interface AutoCloseable {
    void close() throws Exception;
}
Example: FileInputStream, BufferedReader
```

Good Usage: Depend on the Capability You Need

```
public void process(Iterable<String> names) {
    for (String name : names) {
        System.out.println(name);
    }
}
```

We only need the ability to iterate, not anything else about the collection. That's ISP in action!

Key Takeaway

Interfaces in Java are not about being classes with abstract methods.
They are about expressing capabilities.

ISP in One Line

Many small, role-specific interfaces > One large, general-purpose interface.
Better design, flexible code, happy clients!

Design for what a type can do, not for what it is.
That's the power of ISP.

ISP in Framework Design

Interface Segregation Principle in Action

ISP: Clients should not be forced to depend on methods they do not use.
Prefer several small, focused interfaces over one large interface.

Bad Design — One Fat Interface

Every component is forced to implement methods it doesn't need.

```
public interface ApplicationComponent {
    void start();
    void stop();
    void configure();
    void authenticate();
    void log();
    void serialize();
    void cache();
    void monitor();
}
```

Problems

- ✖ Components must implement methods they don't use.
- ✖ Unnecessary dependencies increase complexity.
- ✖ Harder to maintain and test.
- ✖ Changes affect many unrelated components.

Example Components Forced to Implement Everything

SimpleLogger (only logs)

```
public class SimpleLogger
   implements ApplicationComponent {

   // Must implement ALL methods
   // even if not needed

   ... many empty or irrelevant
   implementations ...
}
```

FileCache (only caches)

```
public class FileCache
   implements ApplicationComponent {

   // Must implement ALL methods
   // even if not needed

   ... many empty or irrelevant
   implementations ...
}
```

Result: Tight coupling, unnecessary code, and harder evolution.

Better Design — Many Focused Interfaces

Components depend only on the capabilities they actually need.

Startable

```
public interface
  Startable {
    void start();
}
```

Stoppable

```
public interface
  Stoppable {
    void stop();
}
```

Configurable

```
public interface
  Configurable {
    void configure();
}
```

Authenticatable

```
public interface
  Authenticatable {
    void authenticate();
}
```

Loggable

```
public interface
  Loggable {
    void log();
}
```

Serializable

```
public interface
  Serialicable {
    void serialize();
}
```

Cacheable

```
public interface
  Cacheable {
    void cache();
}
```

Monitorable

```
public interface
  Monitorable {
    void monitor();
}
```

Components Depend Only on What They Need

SimpleLogger (only logs)

```
public class SimpleLogger
  implements Loggable {
   @Override
   public void log() {
      // logging logic
   }
}
```

FileCache (only caches)

```
public class FileCache
  implements Cacheable {
   @Override
   public void cache() {
      // caching logic
   }
}
```

AuthService (authenticates)

```
public class AuthService
  implements Authenticatable {
   @Override
   public void authenticate() {
      // authentication logic
   }
}
```

SystemMonitor (monitors)

```
public class SystemMonitor
  implements Monitorable {
   @Override
   public void monitor() {
      // monitoring logic
   }
}
```

Result: Components are **focused, reusable, easy to maintain,** and **composable.**

Key Takeaway

- Do not create "do-everything" interfaces.
- Provide small, role/capability-based interfaces.
- Clients implement only what they actually need.

Framework Benefit

- Easier for developers to adopt and use.
- Better extensibility through composition.
- Encourages clean architecture and SOLID design.

From Fat Interface to Composable Design

Big Interface (forces everything) → **Segregated Interfaces** (small capabilities) → **Composable Framework** (flexible & clean)

D — Dependency Inversion Principle

A key to flexible, testable, and framework-friendly applications

Definition

1. High-level modules should not depend on low-level modules.
   Both should depend on **abstractions**.
2. Abstractions should not depend on details.
   Details should depend on **abstractions**.

✗ Violates DIP

High-level module depends on low-level module




```
// High-level module
class OrderService {
   private MySQLRepository repository = new MySQLRepository();

   public void placeOrder(Order order) {
      // business logic
      repository.save(order);
   }
}
```

- Tightly coupled
- Hard to test
- Hard to change (e.g., switch database)
- Violates DIP

✓ Follows DIP

Both depend on abstraction




```
// Abstraction
interface OrderRepository {
   void save(Order order);
}
// High-level module depends on abstraction
class OrderService {
   private final OrderRepository repository;

   public OrderService(OrderRepository repository) {
      this.repository = repository;  // Dependency injected
   }

   public void placeOrder(Order order) {
      // business logic
      repository.save(order);
   }
}
```

Benefits

- Loosely coupled
- Easy to test (use mocks)
- Easy to change (swap implementations)
- Aligns with SOLID
- Foundation of frameworks like Spring

Key Idea

Depend on what is stable (abstractions), not on what changes (details).

This keeps your high-level business logic independent of implementation details.

Real-World Example: Spring Framework

1. Define Abstraction

```
public interface EmailSender {
   void send(String to, String msg);
}
```

2. Provide Implementations (Details)

```
@Component
public class SmtpEmailSender implements EmailSender {
   public void send(String to, String msg) {
      // send via SMTP
   }
}
```

```
@Component
public class MockEmailSender implements EmailSender {
   public void send(String to, String msg) {
      // mock for testing
   }
}
```

3. Depend on Abstraction (High-level Module)

```
@Service
public class NotificationService {
   private final EmailSender emailSender;

   // Constructor Injection (DIP in action)
   public NotificationService(EmailSender emailSender) {
      this.emailSender = emailSender;
   }

   public void notifyUser(String user, String msg) {
      emailSender.send(user, msg);
   }
}
```

4. Spring Wiring (Inversion of Control)

```
@Configuration
public class AppConfig {

   @Bean
   public EmailSender emailSender() {
      return new SmtpEmailSender();
      // or return new MockEmailSender();
   }
}
```

spring

Spring inverts the control and injects the implementation.

Your business code depends on abstractions, not on details.

Remember

DIP is the heart of flexible architecture.
It enables plugins, frameworks, IoC containers (like Spring), and ultimately easy-to-maintain, testable systems.

In One Line

High-level modules depend on abstractions.
Details depend on abstractions.
Abstractions do not depend on details.

> **Programs to interfaces, not implementations.**

DIP — Bad Example

High-Level Module depends on a Concrete Implementation

Dependency Inversion Principle

High-level modules should not depend on low-level modules.
Both should depend on abstractions.

Code Example — Tightly Coupled to MySQL

MySQLDatabase.java

```java
public class MySQLDatabase {
    public void save(String data) {
        System.out.println("Saving to MySQL: " + data);
    }

}
```

UserService.java

```java
public class UserService {
    private MySQLDatabase database;

    public UserService() {
        database = new MySQLDatabase();
    }

    public void saveUser(String user) {
        database.save(user);
    }
}
```

Coupling Diagram

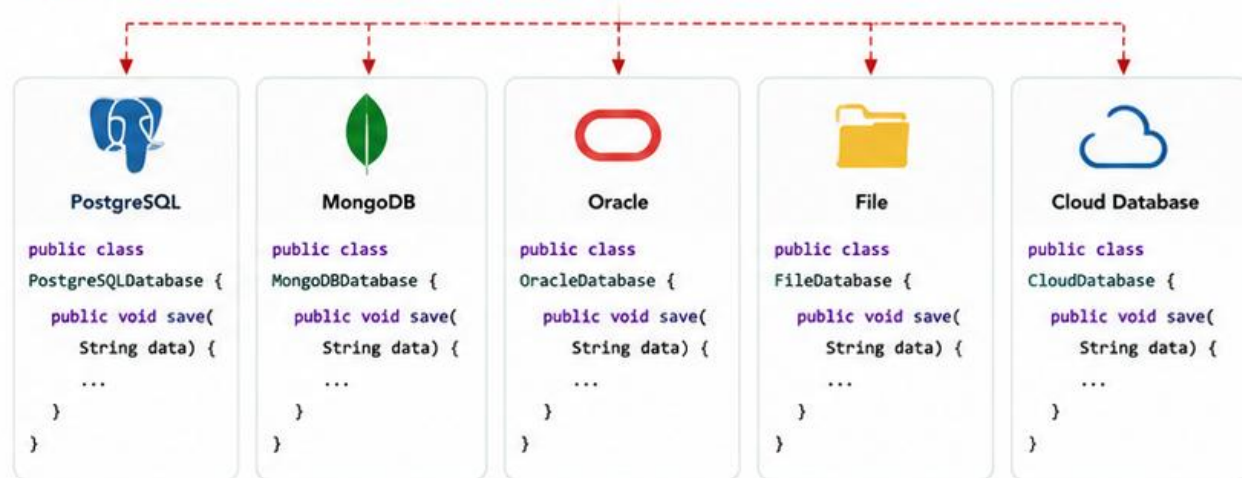


Problem:
UserService is tightly coupled to MySQLDatabase.

What's the Problem?

- ✖ UserService depends on a concrete class (MySQLDatabase).
- ✖ Changing the database requires modifying UserService.
- ✖ Hard to test (cannot easily mock MySQLDatabase).
- ✖ Violates DIP: High-level module should not depend on low-level modules.

What if We Want to Change the Database?



✖ We must modify UserService every time we change the database!

Consequences of This Design

Tight Coupling
Hard to change one part without affecting another.

Hard to Extend
Adding new databases requires code changes in UserService.

Hard to Test
Difficult to mock MySQLDatabase for unit testing.

Violates DIP
Depends on details (concrete class), not on abstractions.

Key Takeaway

Depending on concrete implementations makes the system rigid, hard to maintain, and hard to extend.
DIP helps us depend on abstractions, not on concrete classes.



✖ This is what DIP warns us about.

DIP — Introduce Abstraction

High-level modules depend on abstractions, not on concrete implementations.

Goal: Make **UserService** independent of how users are stored by depending on an abstraction (**UserRepository**).

1. Abstraction

```
public interface UserRepository {
    void save(String user);
}
```

This is the contract.
It defines **what** can be done.

2. MySQL Implementation

```
public class MySQLUserRepository
     implements UserRepository {
    @Override
    public void save(String user) {
        System.out.println("Saving to MySQL");
    }
}
```

3. PostgreSQL Implementation

```
public class PostgreSQLUserRepository
     inplements UserRepository {
    @Override
    public void save(String user) {
        System.out.println("Saving to PostgreSQL");
    }
}
```

4. High-level Module (Depends on Abstraction)

```
public class UserService {
    private final UserRepository repository;
    public UserService(UserRepository repository) {
        this.repository = repository;
    }
    public void saveUser(String user) {
        repository.save(user);
    }
}
```

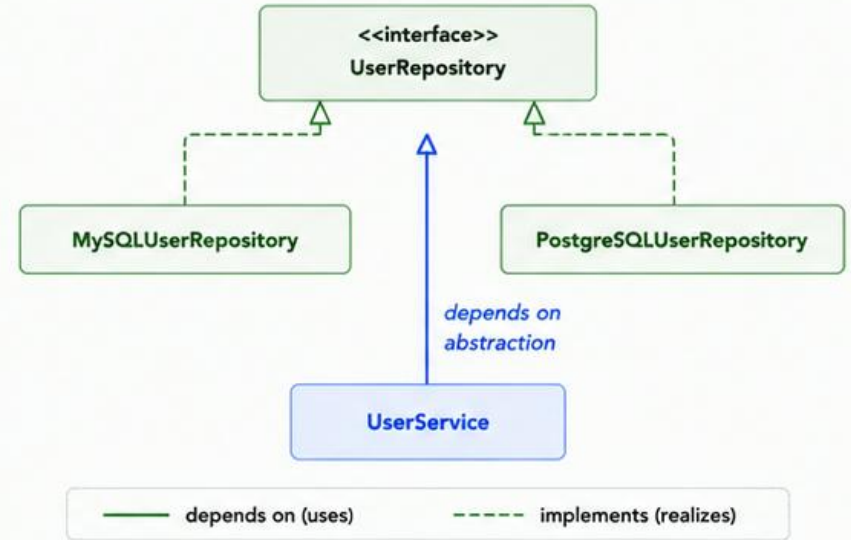
5. Usage (Client Code)

```
// Use MySQL
UserRepository repo1 = new MySQLUserRepository();
UserService service1 = new UserService(repo1);
service1.saveUser("Alice");   // Saving to MySQL
```

or

```
// Use PostgreSQL
UserRepository repo2 = new PostgreSQLUserRepository();
UserService service2 = new UserService(repo1);
service2.saveUser("Bob");    // Saving to PostgreSQL
```

Dependency Structure (After Introducing Abstraction)



What Changed?

- **UserService** no longer depends on **MySQLUserRepository** or **PostgreSQLUserRepository**.
- It depends only on the abstraction: **UserRepository**.
- We can switch implementations without changing **UserService**.

Benefits

- Loose coupling
- Easy to switch database
- Easy to test (use mock of UserRepository)
- Follows **Dependency Inversion Principle (DIP)**

Key Takeaway

High-level modules (like **UserService**) should not depend on low-level modules (like **MySQLUserRepository** or **PostgreSQLUserRepository**).
Both should depend on abstractions (like **UserRepository**).



Depend on what is stable (abstraction), not on what changes (details).

Dependency Injection vs Dependency Inversion

Dependency Inversion (Principle)

High-level modules should not depend on low-level modules.
Both should depend on abstractions.

Dependency Injection (Technique)

A way to implement Dependency Inversion.
Dependencies are provided (injected) from outside.

Without Dependency Injection (Bad)

UserService is responsible for creating its dependency.
Tightly coupled to a concrete implementation.

UserService.java

```java
public class UserService {
    private MySQLUserRepository repository;

    public UserService() {
        repository = new MySQLUserRepository();
    }

    public void saveUser(User user) {
        repository.save(user);
    }
}
```

MySQLUserRepository.java

```java
public class MySQLUserRepository
        implements UserRepository {

    @Override
    public void save(User user) {
        System.out.println("Saving user
            to MySQL database");
    }
}
```

Problems

Tight Coupling
UserService depends on MySQL specifically.

Hard to Change
To use another database, we must modify UserService.

Hard to Test
Cannot easily mock MySQLUserRepository for unit testing.

Violates DIP
High-level module depends on a concrete low-level module.

With Dependency Injection (Good)

The dependency is provided (injected) from outside.
UserService depends on an abstraction.

UserRepository.java (Abstraction)

```java
public interface UserRepository {
    void save(User user);
}
```

MySQLUserRepository.java

```java
public class MySQLUserRepository
      implements UserRepository {

    @Override
    public void save(User user) {
        System.out.println("Saving user
            to MySQL database");
    }
}
```

UserService.java

```java
public class UserService {
    private final UserRepository repository;

    // Dependency is injected via constructor
    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public void saveUser(User user) {
        repository.save(user);
    }
}
```

How Dependency Injection Works

1. Create dependency

```java
UserRepository repository =
    new MySQLUserRepository();
```

2. Inject dependency

```java
UserService service =
    new UserService(repository);
```

3. Use the service

```java
service.saveUser(user);
```

Key Takeaways

- **Dependency Inversion is the principle**: depend on abstractions, not on concretions.
- **Dependency Injection is the technique**: provide the dependency from outside.
- DI makes code more **flexible**, **testable**, and maintainable.

Real-World Analogy

Instead of a class creating its own tools (**new MySQLUserRepository()**), we hand the tools to it from the outside.
The class just uses what it needs, without knowing how it was created.



Constructor Injection

Prefer constructor injection over creating dependencies inside the class.

Principle: Make dependencies explicit so that classes are easier to understand, test, and maintain.

Avoid: Creating Dependency Inside the Class

```
public class UserService {
   private UserRepository repository;

   public UserService() {
      repository = new MySQLUserRepository();
   }

   public void saveUser(String user) {
      repository.save(user);
   }
}
```

Problems

- Dependency is hidden. The class decides what to use.
- Tight coupling to a concrete implementation.
- Hard to change implementation (e.g., MySQL → PostgreSQL).
- Hard to test. You cannot pass a mock or fake.

Prefer: Constructor Injection

```
public class UserService {
   private final UserRepository repository;

   public UserService(UserRepository repository) {
      this.repository = repository;
   }

   public void saveUser(String user) {
      repository.save(user);
   }
}
```

Benefits

- Dependency is explicit and visible.
- Depends on abstraction (DIP).
- Easy to switch implementations.
- Easy to test. Pass a mock or fake.

Testing Becomes Easy

With constructor injection, we can pass a fake or mock implementation.

```
class FakeUserRepository implements UserRepository {
   @Override
   public void save(String user) {
      System.out.println("[FAKE] Saving user: " + user);
   }
}
```

```
UserRepository fakeRepository =
          new FakeUserRepository();

 UserService service =
      new UserService(fakeRepository);

service.saveUser("Alice");
```

Output

```
[FAKE] Saving user: Alice
```

Key Takeaway

Constructor injection makes dependencies **explicit**, promotes **loose coupling**, and greatly improves **testability**.

Rule of Thumb

If a class has dependencies, receive them through the **constructor**.

Best Practice

Use **constructor injection** for mandatory dependencies (final).

Frameworks Love It

Spring and many modern frameworks prefer constructor injection.

DIP and Spring Framework

From Design Principle → Framework in Action

DIP (Principle)

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Spring DI (Technique)

Spring's Dependency Injection container provides dependencies from outside. This implements DIP.

1. Define the Abstraction (Dependency Inversion)

```java
public interface UserRepository {
    void save(String user);
}
```

High-level module depends on this abstraction, not on any concrete class.

2. Provide a Concrete Implementation

```java
@Repository
public class MySQLUserRepository
   implements UserRepository {

   @Override
   public void save(String user) {
      // save user to MySQL database
      System.out.println("Saving user to
      MySQL");
   }

}
```

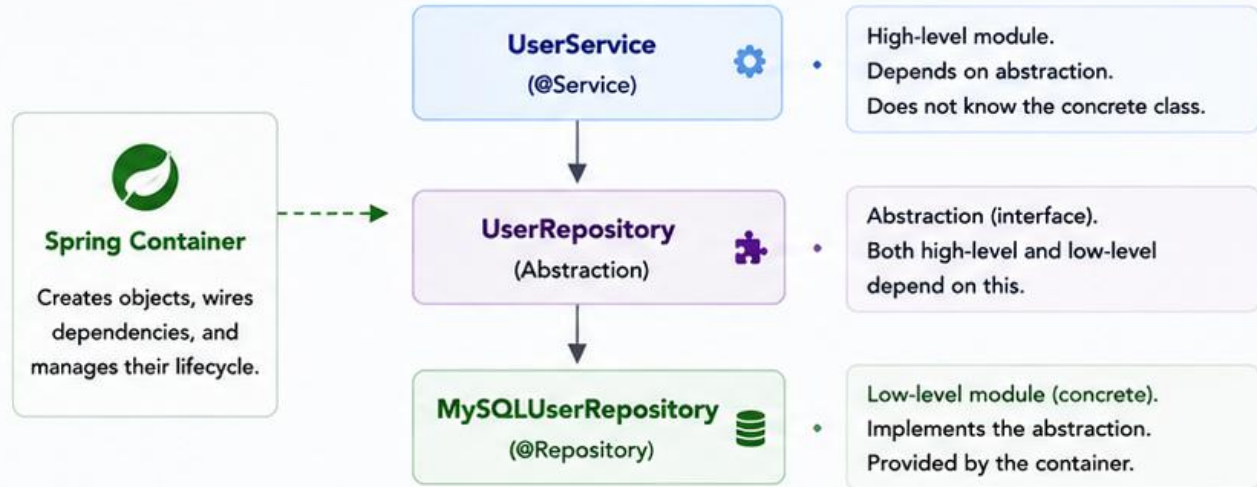
3. Depend on the Abstraction (Constructor Injection)

```java
@Service
public class UserService {
   private final UserRepository repository;

   // Dependency is injected by Spring
   public UserService(UserRepository repository) {
      this.repository = repository;
   }

   public void registerUser(String user) {
      repository.save(user);
   }
}
```

4. Spring Container Manages the Object Graph



Key Idea

You focus on abstractions and business logic. Spring focuses on wiring and lifecycle.

Without Spring (Manual Wiring)

```java
public class App {
   public static void main(String[] args) {
      UserRepository repo = new MySQLUserRepository();     // manual
      UserService service = new UserService(repo);         // manual
      service.registerUser("Alice");
   }
}
```

- Tight coupling.
- Many manual `new`.
- Harder to change implementations.
- Harder to test.

With Spring (Container Wiring)

```java
// No manual wiring!
@SpringBootApplication
public class App {
   public static void main(String[] args) {
      SpringApplication.run(App.class, args);
   }
}
```

- Loose coupling.
- No manual `new`.
- Easy to swap implementations.
- Easy to test (mocks).

What You Achieve

- Follows DIP: Depend on abstractions, not concretions.
- Loosely coupled, maintainable code.
- Easier to extend (add PostgreSQL, MongoDB, etc.).
- Easier to unit test (mock the interface).
- Framework operationalizes SOLID principles.

The framework does not replace SOLID. The framework operationalizes many SOLID ideas.

SOLID is the foundation. Frameworks like Spring help you apply those principles effectively.

SOLID Together — Complete Example

Online Shopping System

Order | Payment | Notification | Persistence | Discount

SOLID turns a fragile, hard-to-change system into a flexible, maintainable, and testable one.

POOR IMPLEMENTATION — Violates SOLID

```
public class OrderProcessor {
  public void process(Order order,
                      String paymentType,
                      String customerType) {
    // 1. calculate discount
    if (customerType.equals("VIP")) {
       // discount
    }
    // 2. payment
    if (paymentType.equals("CARD")) {
       // card payment
    }
    // 3. save to database
    // save order
    // 4. send email
    // send email to customer
  }
}
```

Problems

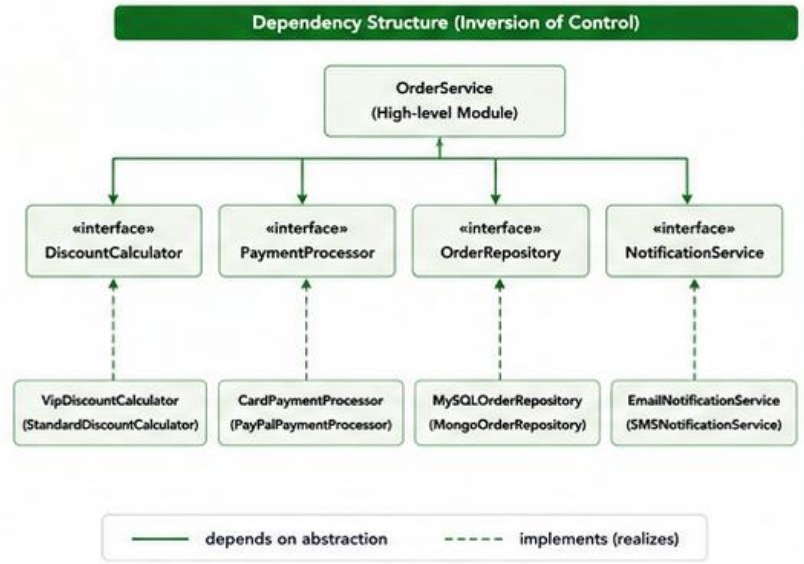
- **Single Responsibility Violation:** Does too many things.
- **Open/Closed Violation:** Hard to add new payment or discount types.
- **Liskov Substitution Violation:** No abstractions, hard to substitute behaviors.
- **Interface Segregation Violation:** Clients forced to depend on methods they don't need.
- **Dependency Inversion Violation:** Depends on concrete details (payment, DB, email, etc.).

SOLID IMPLEMENTATION — Clean, Flexible, and Maintainable

```
// High-level module
public class OrderService {
   private final DiscountCalculator discountCalculator;
   private final PaymentProcessor paymentProcessor;
   private final OrderRepository orderRepository;
   private final NotificationService notificationService;

   public OrderService(DiscountCalculator discountCalculator,
                       PaymentProcessor paymentProcessor,
                       OrderRepository orderRepository,
                       NotificationService notificationService) {
      this.discountCalculator = discountCalculator;
      this.paymentProcessor = paymentProcessor;
      this.orderRepository = orderRepository;
      this.notificationService = notificationService;
   }

   public void placeOrder(Order order, PaymentDetails paymentDetails) {
      double discount = discountCalculator.calculate(order);
      paymentProcessor.processPayment(order, paymentDetails, discount);
      orderRepository.save(order);
      notificationService.sendOrderConfirmation(order);
   }
}
```



S — Single Responsibility Principle

Each class has one reason to change.
OrderService orchestrates only.
Each collaborator has a single responsibility.

```
DiscountCalculator
PaymentProcessor
OrderRepository
NotificationService
```

O — Open/Closed Principle

Open for extension, closed for modification.

Add new payment or discount by creating new implementations, not by changing existing code.

```
+ PayPalPaymentProcessor
+ CryptoPaymentProcessor
+ SeasonalDiscountCalculator
```

L — Liskov Substitution Principle

Subtypes can be substituted for their base types.

Any implementation of the interfaces can be used without breaking correctness.

```
PaymentProcessor p = new CardPaymentProcessor();
p.processPayment(...);      // works!
```

I — Interface Segregation Principle

Many small, specific interfaces instead of one fat interface.

Clients depend only on the methods they actually need.

No client is forced to depend on unrelated methods.

D — Dependency Inversion Principle

High-level modules depend on abstractions, not on details.

Concrete implementations depend on abstractions.

```
OrderService depends on interfaces.
Implementations depend on interfaces.
```

Key Takeaway

SOLID together leads to code that is **easier** to understand, extend, test, and **maintain** — and is the foundation of frameworks like Spring.

Refactored Architecture — SOLID in Action

From Tight Coupling to Flexible, Extensible Design

1 Create Abstractions (Interfaces)

PaymentProcessor

```
public interface
  PaymentProcessor {
   void pay(double amount);
}
```

DiscountPolicy

```
public interface
 DiscountPolicy {
  double calculate(
      double amount);
}
```

OrderRepository

```
public interface
 OrderRepository {
   void save(Order order);
}
```

NotificationService

```
public interface
 NotificationService {
  void send(Order order);
}
```

2 Use Abstractions in High-Level Module

OrderService

```
public class OrderService {
    private final PaymentProcessor payment;
    private final DiscountPolicy discount;
    private final OrderRepository repository;
    private final NotificationService notification;
    public OrderService(
        PaymentProcessor payment,
        DiscountPolicy discount;
        OrderRepository repository,
        NotificationService notification) {
         this.payment = payment;
         this.discount = discount;
         this.repository = repository;
         this.notification = notification;
    }

    public void process(Order order) {
        double total = discount.calculate(order.getAmount());
        payment.pay(total);
        repository.save(order);
        notification.send(order);
    }
}
```

Why this is better

- Depends on abstractions, not concrete classes.
- Easy to change implementations without modifying OrderService.
- Easy to test (mock interfaces).
- Easy to extend (add new implementations).
- Follows multiple SOLID principles.

Before: Tight Coupling (Hard to Change)

```
public class OrderService {
   private CreditCardPayment payment;
   private PercentageDiscount discount;
   private MySQLOrderRepository repository;
   private EmailNotificationService notification;

public void process(Order order) {
    double total = discount.calculate(order.getAmount());
    payment.pay(total);
    repository.save(order);
    notification.send(order);
}
```

- Depends on concrete classes.
- Hard to swap implementations.
- Hard to test (no abstraction).
- Changes ripple into OrderService.

After: Loosely Coupled (Easy to Change)



New implementations can be added without touching OrderService.

3 SOLID Principles Demonstrated

S	Single Responsibility	OrderService orchestrates the workflow. Each collaborator has one responsibility.
O	Open/Closed	Open for extension (new implementations), closed for modification (OrderService).
L	Liskov Substitution	Any implementation can replace another without breaking behavior.
I	Interface Segregation	Focused interfaces (pay, calculate, save, send) — no client is forced to depend on more.
D	Dependency Inversion	High-level module (OrderService) depends on abstractions, not concrete details.

Key Takeaway

By introducing abstractions and injecting dependencies, we achieve a design that is flexible, testable, maintainable, and ready for change — the essence of SOLID design.

Abstractions (Interfaces) → Dependency Injection → Loose Coupling High Cohesion → SOLID Architecture

Identify the Principles

Given the design below, identify how each SOLID principle is applied.

Context: Online Shopping System

We want to place an order. The system needs to calculate discount, process payment, save the order, and send a notification.

OrderService (High-level Module)

```
public class OrderService {
   private final PaymentProcessor paymentProcessor;
   private final DiscountPolicy discountPolicy;
   private final OrderRepository orderRepository;
   private final NotificationService notificationService;

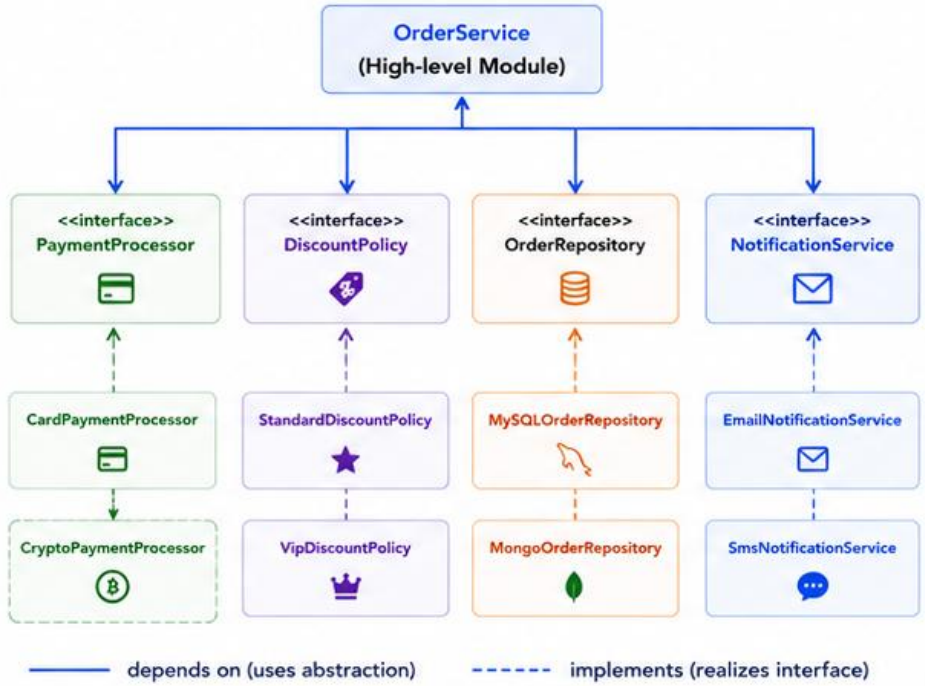
   public OrderService(PaymentProcessor paymentProcessor,
                       DiscountPolicy discountPolicy,
                       OrderRepository orderRepository,
                       NotificationService notificationService) {
      this.paymentProcessor = paymentProcessor;
      this.discountPolicy = discountPolicy;
      this.orderRepository = orderRepository;
      this.notificationService = notificationService;
   }

   public void placeOrder(Order order, PaymentDetails payment) {
     double discount = discountPolicy.calculate(order);
     PaymentResult result = paymentProcessor.processPayment(order, payment);
     order.applyDiscount(discount);
     orderRepository.save(order);
     notificationService.sendOrderConfirmation(order, result);
}
```

Key Observation

`OrderService` coordinates the workflow by using abstractions.
Concrete details are hidden behind interfaces.

Overall Architecture



What This Means

- OrderService does not know the concrete classes.
- We can add new implementations without changing OrderService.
- Every implementation must fulfill its contract.
- Interfaces are small and focused.
- High-level module depends on abstractions, not details.

1 SRP – Single Responsibility Principle

`OrderService` coordinates the process rather than implementing:

- payment
- persistence
- notification
- discount calculation



2 OCP – Open/Closed Principle

We can introduce:

`CryptoPaymentProcessor`

without changing OrderService.

3 LSP – Liskov Substitution Principle

Any valid implementation of:

`PaymentProcessor`

should behave according to its contract.

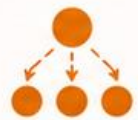
4 ISP – Interface Segregation Principle

Instead of one huge:

`OrderSystemInterface`

we use focused interfaces:

- PaymentProcessor
- DiscountPolicy
- OrderRepository
- NotificationService

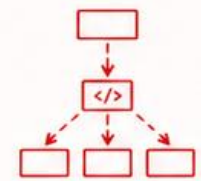


5 DIP – Dependency Inversion Principle

OrderService depends on abstractions:

- PaymentProcessor
- DiscountPolicy
- OrderRepository
- NotificationService

not concrete implementations.



Takeaway

The system is easier to understand, extend, test, and maintain because SOLID principles guide the design.

SOLID and Design Patterns

Design Patterns can help achieve SOLID — but they do not guarantee it.

Important Reminder
A design pattern does not automatically guarantee SOLID compliance.
It depends on how you apply and implement it.

SOLID Principlee and Patterns Commonly Related

SOLID Principle	Patterns Commonly Related
S Single Responsibility Principle (SRP)	Facade, Command, Observer
O Open/Closed Principle (OCP)	Strategy, Decorator, Factory
L Liskov Substitution Principle (LSP)	Template Method, Strategy
I Interface Segregation Principle (ISP)	Adapter, Facade
D Dependency Inversion Principle (DIP)	Factory, Dependency Injection, Abstract Factory

Example: Strategy Pattern Implemented Well vs Implemented Badly

Good Implementation (SOLID-Friendly)

- Each strategy has a single responsibility.
- Strategies are interchangeable (LSP).
- Context depends on abstraction (DIP).

```
// Strategy interface
public interface PaymentStrategy {
   void pay(double amount);
}
// Concrete strategies
public class CreditCardPayment implements PaymentStrategy {
   public void pay(double amount) { /* pay by card */ }
}
public class PayPalPayment implements PaymentStrategy {
   public void pay(double amount) { /* pay by PayPal */ }
}
// Context depends on abstraction (DIP)
public class OrderService {
    private final PaymentStrategy strategy;

    public OrderService(PaymentStrategy strategy) {
       this.strategy = strategy;
    }
    public void checkout(double amount) {
       strategy.pay(amount);
    }
}
```

Easy to extend, test, and maintain. Follows SOLID.

VS

Bad Implementation (Violates SOLID)

- Strategies have multiple responsibilities (violates SRP).
- Hard-coded dependencies (violates DIP).
- Not easily interchangeable (may violate LSP/OCP).

```
// Fat strategy - does too many things
public class PaymentStrategy {
  public void pay(double amount) { /* pay */ }
  public void saveToDatabase() { /* DB logic */ }
  public void sendEmail() { /* email logic */ }
  public void generateReport() { /* reporting */ }
}

// Context creates and depends on concrete class
public class OrderService {
  private final PaymentStrategy strategy = new PaymentStrategy();

  public void checkout(double amount) {
    strategy.pay(amount);
    strategy.saveToDatabase();
    strategy.sendEmail();
    strategy.generateReport();
  }
}
```

Hard to maintain, hard to test, hard to extend. Violates SOLID.

Key Takeaways

Patterns are Tools, Not Silver Bullets
A pattern provides structure, but you must apply it properly to achieve SOLID.

SOLID Guides Good Design
Use SOLID principles to evaluate your design and pattern implementations.

Good Design = Principles + Patterns
SOLID principles give you the "why", patterns give you the "how".

Bad Implementation Can Still Break Everything
Even using the "right" pattern can lead to tight coupling, rigid code, and bugs.

Always Think Before You Code
Understand the problem, apply principles, then choose the pattern.

Golden Rule
Use patterns to support SOLID, not to replace thinking.
SOLID is the foundation. Patterns are the tools.

A design pattern does not make your code SOLID. Your design decisions do. | Think. Design. Then Code.

SOLID → Patterns → Frameworks

From fundamental principles to reusable application infrastructure

The Progressive Flow

SOLID
Design Principles

Fundamental rules that guide good object-oriented design.

Design Patterns
Reusable Design Solutions

Proven, reusable solutions to recurring design problems based on principles.

Frameworks
Reusable Application Infrastructure

Collections of patterns and best practices that provide a reusable application structure.

Architecture
Organized, Maintainable, Scalable Systems

The blueprint of the system that achieves functional and quality requirements.

Another Way to Look at It

Principles — "How should I design?"

↓

Patterns — "How can I solve this recurring design problem?"

↓

Frameworks — "How can I provide a reusable application structure?"

↓

Architecture — A well-structured, maintainable, and scalable system.

Key Takeaway

SOLID principles are the foundation. Patterns are the proven solutions. Frameworks are the practical realization. Together, they lead to sound architecture.

- Principles keep your design right.
- Patterns make your design better.
- Frameworks help you build faster and reuse more.

SOLID Is Not "More Classes"

SOLID is about **managing complexity** and **change** — not creating more types.

Common Student Mistake

Students think SOLID means creating lots of interfaces and classes.
That is unnecessary and harmful for simple problems.

1. Simple Requirement

We just need a calculator that can add two numbers.

Calculator.java

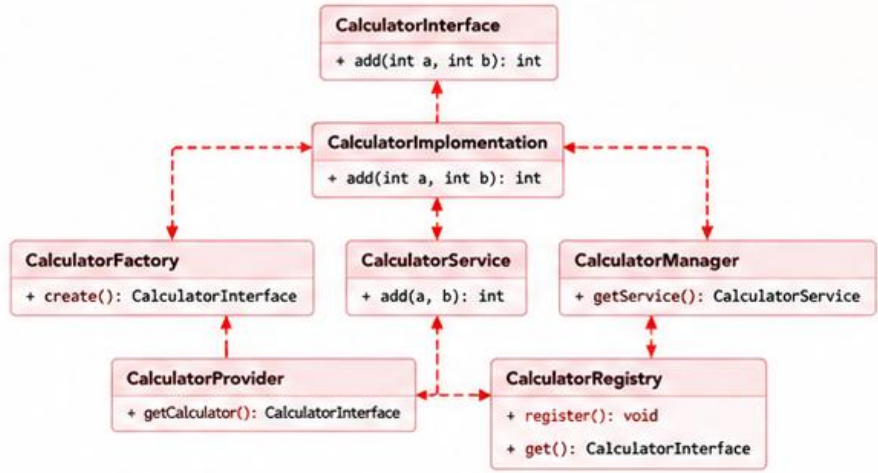
```
class Calculator {
   public int add(int a, int b) {
       return a + b;
   }
}
```

Why this is enough

- **Single responsibility**
  Only one reason to change.
- **Easy to understand**
  Simple, direct, no unnecessary indirection.
- **Easy to test**
  Less code, fewer moving parts.
- **Easy to maintain**
  Change when needed, not before.

2. Over-Engineering (Common Student Trap)

Creating many classes/interfaces for a trivial requirement.



Problems with over-engineering

- Too many classes to read and understand.
- Harder to test (more mocks, more setup).
- More code to maintain with no real benefit.
- Change becomes harder, not easier.
- Violates YAGNI (You Aren't Gonna Need It).

3. The Real Goal of SOLID

SOLID is not about the number of classes.
It is about the right structure for the right reasons.

SOLID Helps You...

Manage Change
Make it easy to change parts that are likely to change.

Reduce Impact
Limit the ripple effect of a change.

Improve Understandability
Code that is easier to read, reason about, and extend.

Deliver Value
Focus on what the system needs today,
not on what might be needed someday.

Rule of Thumb

Start simple. Add abstractions only when:

- You **have multiple reasons to change.**
- You see change happening.
- You need to support variation.
- You want to reduce coupling.

Key Takeaway

SOLID = Right Design for the Right Reasons

Not more classes. Not more interfaces. Not more patterns.
SOLID is about **managing complexity** and **change**.

Remember

"Any fool can write code that a computer can understand.
Good programmers write code that humans can understand.

— Martin Fowler

SOLID Is About Change

Good design minimizes unnecessary ripple effects when requirements change.

Poor Design — Ripple Effect

Requirement Changes
A new requirement arrives.

↓

Which classes change?
We need to find out what is affected.

↓

How many classes?
Many classes are impacted.

↓

How much existing code?
Large portions of existing code may need to change.

↓

How much testing?
Extensive testing is required.

↓

How much risk?
High regression risk and potential bugs.

Good Design — Minimal Ripple

Change
A requirement changes.

↓

Small isolated area
Change is localized.

↓

Few affected classes
Only a few classes are impacted.

↓

Less existing code
Minimal code needs to be modified.

↓

Less testing
Focused testing is enough.

↓

Lower regression risk
Safer changes, more confidence.

SOLID Helps You Achieve This

S — Single Responsibility Principle (SRP)
Each class has one reason to change.
Changes stay in one place.

O — Open/Closed Principle (OCP)
Open for extension, closed for modification.
Add new behavior without changing existing code.

L — Liskov Substitution Principle (LSP)
Subtypes can replace base types.
Behavior remains consistent, changes stay local.

I — Interface Segregation Principle (ISP)
Clients depend only on what they use.
Smaller interfaces mean fewer ripples.

D — Dependency Inversion Principle (DIP)
Depend on abstractions, not concretions.
Easier to swap implementations without ripple effects.

Bottom Line

SOLID is not about writing more code.
It's about making change **easy**, **safe**, and **affordable**.

Remember

The real test of good design is not when everything works, but when **something changes**.

- Change with confidence
- Lower cost of change
- Faster delivery
- Happier developers & happier customers

SOLID and Testing

Good design makes testing easier. SOLID is a key enabler.

Poor Design (Tightly Coupled)

Hard to test — depends on concrete class.

```
public class UserService {
    private MySQLDatabase database =
        new MySQLDatabase();

    public void saveUser(String user) {
        database.save(user);
    }
}
```

Problems

- Tightly coupled to MySQLDatabase
- Cannot easily use a fake or mock
- Tests require real database
- Slow, brittle, hard to isolate
- Hard to change database technology

Testing is difficult, slow, and unreliable.
Change is painful.

Better Design (Loosely Coupled)

Easy to test — depends on abstraction.

```
public class UserService {
    private final UserRepository repository;

    public UserService(UserRepository repository) {
        this.repository = repository;
    }

    public void saveUser(String user) {
        repository.save(user);
    }
}
```

Test Implementation (Fake)

```
public class FakeUserRepository
     implements UserRepository {

 @Override
 public void save(String user) {
     System.out.println("Fake save");
 }

}
```

Using in Test

```
// Arrange
UserRepository repository =
    new FakeUserRepository();

// Act
UserService service =
    new UserService(repository);
 service.saveUser("Alice");

// Assert (verify behavior)
// e.g., verify Fake saved the user
```

Testing is easy, fast, and reliable.
Change is easy.

How SOLID Leads to Testability

DIP

Depend on abstractions (UserRepository), not concretions (MySQLDatabase).

Loose Coupling

High-level module (UserService) is not coupled to low-level details.

Dependency Injection

Dependencies are provided from outside via constructor (injection).

Testability

We can inject fakes/mocks and test in isolation.

Benefits for Testing

- Use fakes or mocks (no real DB, no network)
- Tests are fast and deterministic
- Test one unit in isolation
- Easier to set up and maintain tests
- Confident refactoring
- Better design overall

Key Takeaway

SOLID principles are not just about design elegance — they directly improve **testability**, **maintainability**, and **changeability**.
Good design today = Easier testing and fewer headaches tomorrow.

Faster Tests
No real DB, no external deps.

Reliable Tests
Deterministic and repeatable.

Confident Changes
Refactor with confidence.

SOLID and Clean Architecture

Design for Change. Depend on Abstractions, Not on Details.

The Dependency Rule (Clean Architecture)

"Source code dependencies can only point inward."

High-level policies do not depend on low-level details. Both depend on abstractions.

Why This Matters

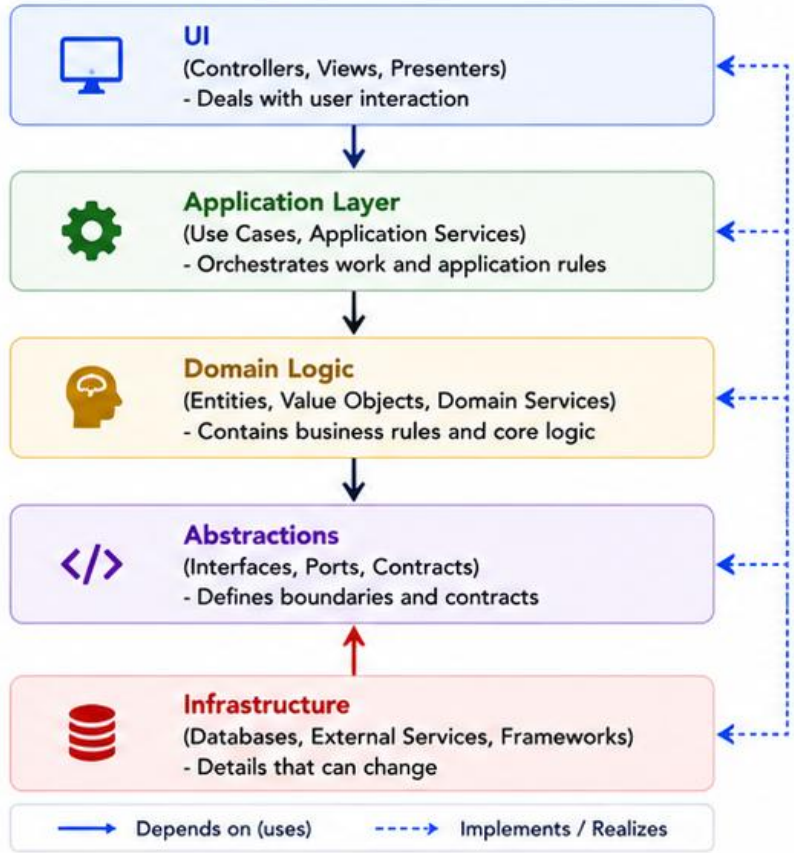
Change Tolerance
Changes in infrastructure (DB, UI, frameworks) do not affect business rules.

Testability
Domain and application logic can be tested without external dependencies.

Independence
Swap infrastructure technologies without changing core logic.

Longevity
Business rules remain stable even as tools and frameworks evolve.

Dependency Direction



How SOLID Supports Clean Architecture

S	**SRP** Single Responsibility Principle	Each layer and class has one reason to change.
O	**OCP** Open/Closed Principle	Extend behavior via new implementations (plugins, policies) without changing existing code.
L	**LSP** Liskov Substitution Principle	Any implementation of an abstraction can be substituted without breaking behavior.
I	**ISP** Interface Segregation Principle	Small, focused interfaces keep dependencies clean and concerns separated.
D	**DIP** Dependency Inversion Principle	High-level modules depend on abstractions. Details depend on abstractions.

Example in Practice

```
// High-level module depends on abstraction
public class OrderService {
   privete final PaymentProcessor paymentProcessor; // interface
   public OrderService(PaymentProcessor paymentProcessor) {
      this.paymentProcessor = paymentProcessor;
   }
}

// Infrastructure implements the abstraction
public class StripePaymentProcessor implements PaymentProcessor { ... }
```

What You Gain

- Protect your core business rules
- Isolate change to the outer layers
- Easier to test at every layer
- Future-proof your architecture

Key Takeaway

SOLID principles help you build **Clean Architecture.**

Clean Architecture enforces the Dependency Rule.

Together, they create systems that are maintainable, testable, and resilient to change.

Remember

Details are volatile.
Abstractions are stable.
Depend on what is stable.

"Make illegal states unrepresentable and dependencies point inward."

FINAL CONCEPT TO TAKE AWAY

Good design is intentional. SOLID helps you think that way.

DON'T MEMORIZE SOLID — ASK THESE QUESTIONS

S — Single Responsibility Principle
Does this class have more than one reason to change?



O — Open/Closed Principle
Can I add new behavior without modifying existing code?



L — Liskov Substitution Principle
Can this subtype safely replace its parent?



I — Interface Segregation Principle
Am I forcing clients to depend on things they don't need?



D — Dependency Inversion Principle
Does my high-level logic depend on concrete implementation details?



FROM SOLID TO BETTER SOFTWARE

SOLID
↓
Better Object Design
↓
Lower Coupling
↓
Higher Cohesion
↓
Easier Testing
↓
Easier Extension
↓
Safer Change

THE BIGGER PICTURE

PRINCIPLES
Fundamental truths that help us evaluate good design.
"What should a good design look like?"

↓

PATTERNS
Proven, reusable solutions to recurring design problems.
"How can I solve this problem well?"

↓

FRAMEWORKS
Reusable application structures and infrastructure.
"How do I apply these ideas consistently at scale?"

Design patterns are not the goal.
Good design is the goal.

SOLID gives us **principles** to evaluate that design.

Patterns give us **reusable solutions** to design problems.

Frameworks help us **apply those ideas** consistently at application scale.

Write code that is easy to change, easy to understand, and a joy to improve. | **THAT IS THE TRUE POWER OF SOLID.**